Jira Test Management Project

Student Assignment Brief for Aspiring Automation Testers

Item	Details
Objective	Plan and manage the complete testing of an online shopping application, starting only from a requirements document
Where the work is done	Entirely inside Jira. No coding and no external application is needed
Tools	Jira Cloud (free plan is enough). Optional: Xray or Zephyr free trial
What you receive	Business requirements, sample product data, and a list of test observations (all in this document)
Duration	1 to 2 weeks
Mode	Individual
Total marks	100

Why this project? Automation testers do far more than write scripts. They read requirements, break them into work items, design test cases, report defects clearly, and show progress through dashboards and reports. This project builds those skills first, so that your automation scripts have a proper home later.

1. The Scenario

You have joined the QA team of a company that is building **ShopEasy**, an online shopping web application. The business team has written the requirements (Section 2). The development team will release version 1.0 soon.

Your job is to take these requirements and, **using only Jira**:

- Write the **Epics** and **User Stories** yourself from the requirements
- Plan the work in **Sprints**
- Write **test cases** for every requirement and link them to your stories
- Record **test execution** results using the observations given in Section 4
- Log **defects** for the observations that do not match the requirements
- Move defects through a QA **workflow**
- Show quality status with a **dashboard**, **filters** and a **summary report**

The chain you must maintain (traceability)

Requirement	User Story	Test Case	Bug
REQ-A1 (given)	Written by you	Written by you	Logged by you
Every requirement needs a story	Every story needs at least one test	Every failed test needs a bug	Every bug links back to its test and story

> **Test environment (fictional):** ShopEasy build 1.0.3, staging server, Chrome latest, Windows 11. Use these details in the Environment field of your bug reports.

Sample product data (for your test cases)

Product	Category	Price	Stock
Cotton Shirt	Clothing	Rs. 1,200	In Stock
Running Shoes	Footwear	Rs. 3,500	In Stock
Water Bottle	Accessories	Rs. 450	In Stock
Backpack	Accessories	Rs. 999	Out of Stock
Notebook	Stationery	Rs. 150	In Stock

2. Business Requirements (Given)

These requirements are the only source of truth. If something is not stated, do not assume it. Currency is Indian Rupees (Rs.).

Module A: Account and Login

ID	Requirement
REQ-A1	Registration: a user registers with full name, email and password. The email must be valid and unique. The password must have at least 8 characters, with at least one number and one special character.
REQ-A2	Login: a registered user logs in with email and password and lands on the home page. Invalid details show the message "Invalid email or password".
REQ-A3	Account lock: after 5 consecutive failed login attempts the account is locked for 15 minutes.
REQ-A4	Forgot password: a reset link is emailed. The link is valid for 30 minutes and can be used only once.
REQ-A5	Logout: logout ends the session. Pressing the browser Back button must not show account pages.

Module B: Browse and Search

ID	Requirement
REQ-B1	Product list: shows 12 products per page with pagination. Each product card shows image, name and price.
REQ-B2	Search: users search by keyword in the product name. Search is not case-sensitive. If nothing matches, the message "No products found" is shown.
REQ-B3	Filters: users can filter by category and by price range (minimum and maximum). Both filters can be used together.
REQ-B4	Sorting: price low to high, price high to low, and name A to Z.
REQ-B5	Product details: shows images, name, price, description and stock status (In Stock or Out of Stock). The Add to Cart button is disabled when the product is out of stock.

Module C: Cart and Checkout

ID	Requirement
REQ-C1	Add to cart: the cart icon shows the total number of items in the cart.
REQ-C2	Quantity: each item can have a quantity from 1 to 10. Changing the quantity updates the line total.
REQ-C3	Remove item: the item is removed and the cart total updates immediately. An empty cart shows a message.
REQ-C4	Coupon: code SAVE10 gives 10% discount on a cart value of Rs. 1,000 or more. Only one coupon per order.
REQ-C5	Checkout: login is required (no guest checkout). Shipping details are all mandatory: name, phone (10 digits), address, city and pin code (6 digits).
REQ-C6	Payment and confirmation: payment by Card, UPI or Cash on Delivery. After placing the order, a confirmation page shows a unique Order ID and a confirmation email is sent.

3. Tasks to Complete (All in Jira)

Complete the tasks in order. Each task builds on the previous one.

Task 1: Create the Jira Project

1. Sign up for a free Jira Cloud site at atlassian.com if you do not have one.
2. Click **Create project** and choose the **Scrum** template.
3. Name it **ShopEasy QA** with the key **SEQ**.
4. Open **Project settings** and confirm the issue types Epic, Story, Task, Bug and Subtask are available.
5. Add a custom issue type named **Test** (skip this if you use Xray or Zephyr).

Output: A working project with a Backlog and a Sprint board.

Task 2: Write Epics and User Stories

What to do: Read all requirements and write the epics and stories yourself. Nothing is given to you here, because turning requirements into work items is a core QA skill.

- Create **at least 3 Epics**. Each epic is a large feature area (for example, one per module).
- Under each epic write **4 to 5 User Stories**, in the form: *As a [user], I want [goal], so that [benefit]*.
- Every story needs at least **two acceptance criteria**, written as: *Given [situation], when [action], then [result]*.
- Put the requirement ID (for example REQ-A2) in the story description and add it as a **label**, so every requirement is traceable.
- Every requirement REQ-A1 to REQ-C6 must be covered by at least one story.

Worked example (do not copy, write your own)

Field	Example
Epic	User Account and Login
Story	As a registered customer, I want to log in with my email and password, so that I can see my account
Acceptance criteria	1. Given I am on the login page, when I enter a valid email and password, then I land on the home page. 2. Given I am on the login page, when I enter a wrong password, then I see "Invalid email or password".
Label / Points	REQ-A2 / 3 story points

Output: 3 or more Epics and 12 to 15 Stories, each linked to its Epic, with acceptance criteria and labels.

Task 3: Plan and Run Two Sprints

1. Give every story **story points** (use 1, 2, 3, 5, 8). Points show relative effort, not hours.
2. Create **Sprint 1** and **Sprint 2**, each one week long (or shorter if your trainer says so).
3. Divide the stories between the sprints. Put the most important and risky stories first.
4. Start Sprint 1 and move cards across the board as you finish writing and running tests for them.
5. Complete Sprint 1, then start and complete Sprint 2.

Output: Two completed sprints with story points, so the Burndown chart has real data.

Task 4: Write Test Cases

What to do: Write at least **25 test cases** as Jira issues of type Test. Each one must have:

- **Summary:** a short title, such as *Verify login fails with a wrong password*
- **Preconditions:** what must be true before you start
- **Steps:** numbered actions, and **test data** (use the sample products where needed)
- **Expected result:** taken from the requirement, never guessed
- **Priority and labels:** for example High, and positive, negative or edge
- **Link:** connect to the user story it covers, using the link type *tests* or *relates to*

Required coverage

Type	Meaning	Minimum	Example
Positive	Valid input, expected to work	10	Register with a valid email and strong password
Negative	Invalid input, expected to fail safely	10	Register with a password that has no special character
Edge / Boundary	Limits and unusual input	5	Quantity 1, 10 and 11; cart value Rs. 999 and Rs. 1,000 with SAVE10

Sample test case

Field	Value
Summary	Verify the account is locked after 5 failed login attempts
Linked story / Requirement	Your story for account lock / REQ-A3
Preconditions	A registered user exists and is not locked
Steps	1. Open the login page 2. Enter the registered email with a wrong password 3. Repeat 5 times in a row
Expected result	After the 5th failed attempt the account is locked for 15 minutes
Type / Priority	Negative / High

> **Coverage rule:** Every requirement must have at least one positive and one negative or edge test. Think about boundaries in the numbers: 8 characters, 5 attempts, 30 minutes, 10 items, Rs. 1,000.

Task 5: Create a Test Plan and Execute Tests

1. **With Xray or Zephyr:** create a *Test Plan* and a *Test Execution*, add all your tests, and set Pass or Fail.
2. **Without a plugin:** add a custom field *Execution Status* (Not Run, Pass, Fail, Blocked) to the Test issue type. Create a Task named *Test Plan - ShopEasy 1.0* that lists scope, approach, environment, schedule and risks.
3. Run your tests using the **Test Observations in Section 4**. Compare each observation with the requirement.
4. If the observed result matches the requirement, the related test is a **Pass**. If it does not, the test is a **Fail**.
5. All tests that do not relate to any observation are treated as **Pass**.
6. For every failed test, log a bug (Task 6) and link it to the failed test.

Output: All your tests with a status, and a test plan issue.

Task 6: Log Defects

What to do: Log a bug for each observation that does not match the requirements. You must log **at least 5 bugs**. A good bug report lets a developer reproduce the problem without asking questions.

Field	What to write
Summary	One line: what is wrong and where
Environment	Build 1.0.3, staging server, Chrome latest, Windows 11
Steps to reproduce	Numbered, exact steps, based on the observation
Expected result	What the requirement says (quote the REQ ID)
Actual result	What the observation says happened
Severity	Impact on the system: Critical, Major, Minor, Trivial
Priority	Urgency to fix: Highest, High, Medium, Low
Attachment	Screenshot of the observation row from this brief, as evidence
Links	Link to the failed test and the related user story

Tip: Think about severity and priority yourself. Not every bug deserves the same values, and your reasoning is marked.

4. Test Observations (Given)

Imagine you ran your tests on ShopEasy build 1.0.3 and noted down what happened. These are the observations. Compare each one with the requirements to decide whether it is a defect.

ID	Module	What the tester did	What was observed
OBS-1	A	Registered a new user with password **abc12345**	Account was created and a success message appeared
OBS-2	A	Entered a wrong password 5 times in a row, then waited	Account locked message appeared. Login worked again after 5 minutes
OBS-3	A	Requested a password reset and opened the link 45 minutes later	Reset page opened and the password was changed
OBS-4	B	Searched for **shirt**, then for **SHIRT**	"shirt" showed Cotton Shirt. "SHIRT" showed "No products found"
OBS-5	B	Sorted by price high to low (products at Rs. 450, 999, 1,200 and 3,500)	Order shown: 999, 450, 3,500, 1,200
OBS-6	C	Set the quantity of Running Shoes to 11	Quantity was accepted and the line total updated
OBS-7	C	Added 2 Notebooks (total Rs. 300) and applied SAVE10	Discount of Rs. 30 was applied
OBS-8	C	Placed an order paying by UPI	Page said "Thank you for your order" with no Order ID. Email was received

Think before you log. Read the exact wording of each requirement, including numbers and conditions. Some observations may look wrong at first glance but match the requirement. Explain your decision for each one in a comment on the related test.

Task 7: Customise the Bug Workflow

A workflow is the set of statuses an issue moves through. Change the Bug workflow to this QA lifecycle:

Status	Meaning	Who moves it here
Open	Bug is reported	Tester
In Progress	Developer is fixing it	Developer
Fixed	Developer says the fix is done	Developer
Retest	Tester is verifying the fix	Tester
Closed	Fix verified, bug is gone	Tester

Open **Project settings > Workflows** (or edit the board columns in a team-managed project), add the statuses Fixed and Retest, and connect the transitions. Then move your bugs through the cycle yourself, playing both tester and developer. Send at least one bug back from Retest to In Progress with a comment explaining why the fix failed.

Output: A workflow screenshot, and bugs whose history shows every status.

Task 8: Build the Dashboard

Create a dashboard named **ShopEasy QA Dashboard** with these gadgets:

Gadget	What it shows	Configure with
Sprint Burndown	Work remaining over time	Your project board and Sprint 1 or 2
Pie Chart	Defects by status or priority	Filter: issuetype = Bug
Created vs Resolved	Bugs found against bugs fixed	Filter: issuetype = Bug
Filter Results	List of open critical bugs	Filter from Task 9
Test execution results	Pass / Fail / Not Run count	Pie chart on Execution Status, or the Xray / Zephyr gadget

Output: A dashboard with at least 4 working gadgets, shared with view access.

Task 9: Create Saved Filters with JQL

JQL (Jira Query Language) lets you search issues precisely. Save at least **5 filters**. Examples (replace SEQ with your project key):

Filter name	JQL
All open bugs	project = SEQ AND issuetype = Bug AND statusCategory != Done
Open high-priority bugs	project = SEQ AND issuetype = Bug AND priority in (Highest, High) AND status != Closed
Bugs by assignee	project = SEQ AND issuetype = Bug ORDER BY assignee
Failed tests	project = SEQ AND issuetype = Test AND "Execution Status" = Fail
Stories in Sprint 1	project = SEQ AND issuetype = Story AND sprint = "SEQ Sprint 1"

Sprint names depend on your project, so check the exact name in your board.

Task 10: Prepare the Test Summary Report

Write a 1 to 2 page report (PDF or Word) that tells a manager whether ShopEasy 1.0 is ready to release. Include:

- Project overview, scope, environment and duration
- Number of epics, stories, sprints and test cases
- Execution summary table (Total, Passed, Failed, Blocked, Not Run)
- Defect summary by severity and by status
- Key metrics (formulas below)
- Dashboard screenshot
- Risks, observations and a final recommendation (Go or No-Go) with reasons

Metrics and formulas

Metric	Formula	Example
Pass percentage	Passed tests / Executed tests x 100	20 / 27 x 100 = 74%
Fail percentage	Failed tests / Executed tests x 100	7 / 27 x 100 = 26%
Defect density	Total defects / Number of user stories (or test cases)	6 / 14 = 0.43 per story
Requirement coverage	Requirements with at least one test / Total requirements x 100	17 / 17 x 100 = 100%
Defect closure rate	Closed bugs / Total bugs x 100	5 / 6 x 100 = 83%

5. Deliverables Checklist

#	Deliverable	Format
1	Jira project link with view access for your trainer	URL
2	Screenshot of the ShopEasy QA Dashboard	PNG or JPG
3	Screenshot of the customised bug workflow	PNG or JPG
4	Exported list of test cases and results	CSV or Excel (Jira export)
5	Test Summary Report	PDF or Word

6. Evaluation (100 Marks)

Area	Marks	What the trainer checks
Project and sprint setup	20	Scrum project, 3+ self-written epics, 12+ stories with acceptance criteria, labels and points, 2 completed sprints
Test cases and linking	25	25+ tests, good mix of positive, negative and edge, expected results match requirements, every test linked to a story
Defect quality and lifecycle	25	5+ bugs with all fields, sensible severity and priority, correct judgement of observations, bugs moved through all statuses
Dashboard and filters	20	4+ working gadgets, 5+ saved JQL filters
Summary report	10	Correct metrics, clear Go or No-Go conclusion, professional presentation
Total	**100**	

7. Suggested Timeline

Day	Activity
1	Project setup, read requirements, write epics and stories (Tasks 1 and 2)
2	Story points and sprint planning (Task 3)
3 to 4	Write all test cases and link them (Task 4)
5 to 6	Test plan, execution and defect logging (Tasks 5 and 6)
7	Customise the workflow and move bugs through the lifecycle (Task 7)
8	Dashboard and filters (Tasks 8 and 9)
9 to 10	Test summary report, review and submission (Task 10)

8. Tips and Common Mistakes

- **Do not confuse severity and priority.** Severity is how badly the bug hurts the system. Priority is how soon it must be fixed.
- **Do not assume.** If a requirement does not say something, you cannot call it a defect. Note it as a question or risk instead.
- **One bug, one report.** Do not combine several problems in a single ticket.
- **Use issue keys.** Mention keys like SEQ-12 in comments so related work is easy to find.
- **Keep the board updated daily.** Real teams rely on the board to see progress.
- **Work on your own.** Copied projects are easy to spot because wording and structure look identical.
- **Plugins are optional.** A custom Test issue type with a custom field earns full marks if done well.

> **Looking ahead:** In the next phase of the course you will automate these same test cases and link the results back to Jira. The better your test cases are today, the easier that will be.